Trip Planner Agent - Revised System Design

Sep 24, 2026 · @gopi

What Changed and Why

The original design correctly identifies the major building blocks (frontend, agent engine, tool connectors) but treats several critical decisions as implicit or deferred. This revision makes the following explicit:

- **Trip state is split from conversation memory** — the itinerary needs to be a durable, structured object, not something re-derived from chat history each turn.
- **Tool rollout is phased** — Places and Weather are cheap/free and ready now; Flights and Hotels require partner approval and paid access, so they are deferred to Phase 2.
- **Failure modes are designed for**, not left to happen at demo time.
- **Cost and latency guardrails** are specified upfront (max tool iterations, streaming strategy for tool calls).
- **Auth and persistence** are resolved as a decision rather than left open, since it changes the backend shape significantly.
- **Stack choice is committed**: Vercel AI SDK, not "LangChain or Vercel AI SDK."

Revised Architecture

The main addition is a **Trip State Store** sitting alongside the LLM, separate from conversation memory. The agent reads/writes structured trip data here; it does not rely on the LLM re-deriving the itinerary from chat history each turn.

```
flowchart TD
    User([User]) -->|Chat UI| Frontend

    subgraph NextJS["Next.js Application"]
        Frontend["Frontend: React UI"]
        API["Backend: Route Handlers"]
        Frontend <-->|REST / SSE| API
    end

    subgraph Agent["Agentic Backend (Vercel AI SDK)"]
        Engine["Agent Loop (ReAct)"]
```

```
        LLM["LLM Provider"]
        ConvoMem["Conversation Memory<br/>(message history)"]
        TripState["Trip State Store<br/>(Postgres/Supabase)"]
        Engine <--> LLM
        Engine <--> ConvoMem
        Engine <--> TripState
    end

    subgraph Phase1["Phase 1 Tools (free/cheap)"]
        Places["Places API"]
        Weather["Weather API"]
    end

    subgraph Phase2["Phase 2 Tools (paid/partner)"]
        Flights["Flights API"]
        Hotels["Hotels API"]
    end

    API <--> Engine
    Engine --> Places
    Engine --> Weather
    Engine -.-> Flights
    Engine -.-> Hotels
```

Key differences from the original diagram: trip state is a first-class, persisted component rather than a generic "Memory" box, and Phase 2 tools are drawn as dashed/deferred rather than equal peers of Phase 1 tools.

Trip State Model

The agent's final synthesis step should target a strict schema (via the AI SDK's `generateObject` or typed tool args) rather than freeform text parsed after the fact. Proposed shape:

```
Trip {
  id: string
  userId: string | null        // null for anonymous/session-only trips
  destination: string
  startDate: date
  endDate: date
  budget: number | null
  status: "draft" | "confirmed"
```

```
  days: Day[]
}

Day {
  date: date
  stops: Stop[]
}

Stop {
  time: string | null
  name: string
  type: "attraction" | "meal" | "flight" | "hotel" | "other"
  location: { lat: number, lng: number } | null
  notes: string | null
  sourceToolCallId: string | null   // traceability back to the tool call
that produced this
}
```

The Itinerary View component renders directly off this object, not off parsed chat text. Each tool call that contributes a stop stamps `sourceToolCallId`, so a later failed/retried tool call can be reconciled without corrupting the rest of the itinerary.

Phased Tool Rollout

Phase	Tool	Access	Notes
1	Places API	Free tier (Google Places)	Ready to build now
1	Weather API	Free tier (OpenWeather)	Ready to build now
2	Flights API	Partner approval + paid (Amadeus/Skyscanner)	Defer until there's a booking business case
2	Hotels API	Partner approval + paid (Booking.com)	Defer until there's a booking business case

Phase 1 alone is enough to demo the core loop: understand request → look up places/weather → synthesize itinerary. Phase 2 tools introduce real money movement and partner contracts, so they should be scoped as a separate workstream rather than built alongside Phase 1.

Error Handling and Tool Failure UX

- **Tool API failure (5xx, timeout, rate limit)**: agent catches the error, does not fabricate a substitute result, and either retries once with backoff or surfaces a specific gap to the user ("couldn't get weather for Paris right now, continuing without it") rather than silently omitting it.
- **No results for a valid query**: treated as a distinct case from a failure — the agent says so explicitly rather than inventing plausible-sounding attractions.
- **Malformed/missing tool call args from the LLM**: validated against the tool's schema before execution; a validation failure triggers a re-ask to the LLM with the specific error, not a raw exception to the user.
- **Partial itinerary on failure**: since trip state is a persisted object (see Trip State Model), a failed tool call for Day 2 should not block or corrupt Day 1 — failures are scoped per stop/day, not per whole-trip generation.

Cost, Latency, and Iteration Controls

- **Max tool-call iterations per turn**: cap the ReAct loop (e.g. 5 iterations) to guard against runaway tool-calling loops; on hitting the cap, synthesize with whatever data is available and note what's missing.
- **Streaming strategy**: show tool calls as they happen ("Checking flights to Paris...") rather than hiding the agent behind a single spinner until final synthesis — this is a perceived-latency win, not just a cosmetic one, since multi-tool turns can take several seconds.
- **Caching**: cache Places/Weather responses per destination for a short TTL (e.g. 1 hour) to cut redundant calls when a user iterates on the same trip.
- **Per-turn budget**: a soft ceiling on LLM calls + tool calls per user turn, logged, so cost is visible before Phase 2's paid APIs are added.

Auth and Persistence

Open decision the original design left unresolved — recommendation below, flagged for confirmation:

- **v1 (recommended)**: anonymous, session-scoped trips (a `userId` of `null` in the Trip State Model), stored server-side keyed by a session/browser id so refresh doesn't lose state, but not tied to an account. No auth system to build for the demo.

- **v2**: add accounts (e.g. NextAuth) once there's a reason to persist trips across devices or make them shareable — this changes the backend to require a `users` table and auth middleware on the API routes, and turns `userId` from always-null into a real foreign key.

Stack Decision

Vercel AI SDK (`ai` package), not LangChain. Rationale: this is already a Next.js App Router project, and the AI SDK is built specifically for that combination — native streaming helpers, typed tool calling, and route-handler integration — without LangChain's extra abstraction layer, which mainly earns its keep in more complex multi-agent or multi-vector-store systems this project doesn't need.